

## PostgreSQL

### Ejercicios para practicar

```
CREATE TABLE clientes (  
    id_cliente SERIAL PRIMARY KEY,  
    nombre VARCHAR(100),  
    email VARCHAR(100),  
    ciudad VARCHAR(50),  
    fecha_registro DATE  
);
```

```
CREATE TABLE pedidos (  
    id_pedido SERIAL PRIMARY KEY,  
    id_cliente INT REFERENCES clientes(id_cliente),  
    producto VARCHAR(100),  
    cantidad INT,  
    fecha_pedido DATE  
);
```

#### -- 2. Insertar datos en clientes

```
INSERT INTO clientes (nombre, email, ciudad, fecha_registro) VALUES  
( 'Ana Torres', 'ana.torres@email.com', 'Córdoba', '2025-08-01'),  
( 'Luis Gómez', 'luis.gomez@email.com', 'Rosario', '2025-08-03'),  
( 'María López', 'maria.lopez@email.com', 'Mendoza', '2025-08-05'),  
( 'Carlos Ruiz', 'carlos.ruiz@email.com', 'Salta', '2025-08-07'),  
( 'Sofía Díaz', 'sofia.diaz@email.com', 'Buenos Aires', '2025-08-09');
```

#### -- 3. Insertar datos en pedidos

```
INSERT INTO pedidos (id_cliente, producto, cantidad, fecha_pedido) VALUES  
(1, 'Teclado mecánico', 2, '2025-08-10'),  
(2, 'Mouse inalámbrico', 1, '2025-08-11'),  
(3, 'Monitor 24"', 1, '2025-08-12'),
```

## PostgreSQL

```
(4, 'Auriculares Bluetooth', 3, '2025-08-13'),  
(5, 'Webcam HD', 1, '2025-08-14');
```

-- 4. Consulta combinada

```
SELECT p.id_pedido, c.nombre, p.producto, p.cantidad, p.fecha_pedido  
FROM pedidos p  
JOIN clientes c ON p.id_cliente = c.id_cliente;
```

### Left Joins

```
SELECT clientes.nombre, pedidos.producto  
FROM clientes  
LEFT JOIN pedidos ON clientes.id_cliente = pedidos.id_cliente;
```

Qué hace: Muestra todos los clientes. Si alguno no tiene pedidos, producto será NULL. Útil para: Detectar clientes sin actividad.

### RIGHT JOIN

```
SELECT clientes.nombre, pedidos.producto  
FROM clientes  
RIGHT JOIN pedidos ON clientes.id_cliente = pedidos.id_cliente;
```

Qué hace: Muestra todos los pedidos. Si el cliente fue borrado o no existe, nombre será NULL. Útil para: Auditar pedidos huérfanos o inconsistencias.

### CROSS JOIN

```
SELECT clientes.nombre, pedidos.producto  
FROM clientes  
CROSS JOIN pedidos;
```

Qué hace: Combina cada cliente con cada pedido. Resultado: Si hay 5 clientes y 5 pedidos, devuelve 25 filas. Útil para: Simulaciones, testing, o generar combinaciones posibles.

PostgreSql